

Generation Plan — Sprint Summary

What we built

The **Generation Plan** is LevelUp's signature pre-generation intelligence screen.

When a customer clicks **Generate Script**, we no longer generate immediately. Instead:

1. A brief **"Analyzing repository"** sequence plays (Analyzing Repository → Finding Existing Automation → Evaluating Coverage → Preparing Generation Plan).
2. We render a **Generation Plan** — not a review dialog — that answers the three questions every trustworthy AI feature must:
 - **What did I analyze?** — Existing Automation (covered flows) + Repository Assets Reused.
 - **What did I decide?** — Decision (SKIP / EXTEND / GENERATE), Repository Coverage %, Confidence %, Estimated Token Savings, and the list of Automation to Generate.
 - **Why?** — the `generatedBecause[]` rationale from the frozen policy.
3. The customer approves with **Execute Generation Plan** (or goes Back). When the repository is fully covered (SKIP), execution is disabled — "Nothing to Generate — Fully Covered".

The screen is the product's differentiator versus a raw ChatGPT/Copilot prompt: it proves we read the repo, reused what already exists, and only spend tokens on the gap.

Architecture — the backend stays frozen

The intelligence contract is **frozen**. This sprint added only NEW presentation-layer code; no frozen module was modified.

Layer	File	Role
Backend adapter (NEW)	<code>src/requirement-intelligence/generation-plan-view.ts</code>	Pure, deterministic presentation adapter. Turns the frozen <code>ScriptGenerationPlan</code> + <code>RequirementIntelligence</code> + repo <code>CoverageModel</code> into the exact shape the screen renders. Owns NO intelligence — never re-derives a decision, never matches, never calls an LLM.
Backend route (NEW)	<code>POST /api/scripts/plan</code> in <code>src/api/routes/script-gen.ts</code>	Read-only. Resolves test cases, loads the repo coverage model, runs the frozen <code>RequirementIntelligenceService</code> + <code>ScriptGenerationConsumer</code> , then <code>buildGenerationPlanView(...)</code> . Generates nothing. Falls back to a GENERATE-all view when there is no coverage model / no test cases.
Frontend proxy (NEW)	<code>app/api/scripts/plan/route.ts</code>	Forwards to the backend, carrying session cookie + project/environment/sprint headers.
Frontend UI (NEW)	<code>app/scripts/_components/generation-plan.tsx</code>	<code>GenerationPlanPanel</code> — the analyzing sequence, metric tiles, expandable covered-flow rows, assets-reused list, decision narrative, savings comparison, Back / Execute buttons.
Frontend wiring	<code>app/scripts/_components/script-generator.tsx</code>	“Generate Script” now runs <code>handlePlan()</code> (min ~1.9s animation) → renders the plan → Execute Generation Plan calls the existing generate flow.

The frozen contract this renders

```
{ decision: "EXTEND",
  analysis: { confidence, coveredFlows, missingFlows },
  generatedBecause: [...] }
```

`analysis` = the evidence; `generatedBecause` = why the policy decided. The adapter maps covered behaviors → CoverageModel flows' `testFiles` for per-flow assets, and surfaces the matched feature's `pageObjects` + `helpers` as **Repository Assets Reused** — assets already tied to covered flows, not a new reuse engine.

Honesty guarantees

- Token figures are **estimates** (`ESTIMATED_TOKENS_PER_FLOW × scripts`) and are labeled as estimates in the UI — never presented as measured values.
- The adapter is presentation-only and deterministic; it cannot inflate coverage or fabricate reused assets — every asset shown is one the Coverage Model already attributes to a covered flow.

Validation

- Backend `npx tsc --noEmit` — **exit 0**.
- Frontend `npx tsc --noEmit` — **exit 0**.
- `tests/unit/generation-plan-view.test.ts` — **8/8 passing** (EXTEND, SKIP, GENERATE paths: covered-flow assets, missing flows, reused page objects + helpers, savings comparison, narrative-is-“what”-not-“why”).
- Pre-existing unrelated failures (`execution-provider.test.ts` empty suite, `test-coverage-engine.test.ts` standalone `process.exit`) were not touched by this sprint.